

AtendSaúde

Revisão de Documentação Técnica

Desafio Técnico Full Stack — Seleção V-Lab (CIn-UFPE)

Documento gerado automaticamente por `generate_docs.py` a partir de `/docs` — revise e ajuste livremente antes de exportar em PDF.

1. Modelo de Dados

Modelo de Dados — AtendSaúde

Diagrama Entidade-Relacionamento (ERD)

SOLICITACOES é a entidade central do domínio. PROTOCOLO_SEQUENCES é uma tabela de apoio (contador atômico por ano) — não há chave estrangeira entre as duas: a ligação é **funcional** (a Action de criação lê/incrementa o contador para gerar o campo protocolo), não relacional.

► Diagrama Mermaid — visualize em mermaid.live, num Markdown viewer compatível (ex.: GitHub, VS Code) ou renderize antes de exportar o PDF final.

```
erDiagram
    SOLICITACOES {
        uuid id PK
        string protocolo UK "SOL-{ano}-{sequencial}"
        string nome_solicitante
        enum categoria "CONSULTA | EXAME | VACINACAO | OUTRO"
        enum prioridade "BAIXA | MEDIA | ALTA | URGENTE"
        enum status "RECEBIDA | EM_ANALISE | AGENDADA | CONCLUIDA | CANCELADA"
        text descricao
        text justificativa_prioridade "NULL, exceto se prioridade = URGENTE"
        timestampz data_criacao
        timestampz data_atualizacao
    }

    PROTOCOLO_SEQUENCES {
        int ano PK
        int ultimo_numero
    }
```

Dicionário de Dados

Campo	Tipo (Postgres)	Restrições	Observação
id	UUID	PRIMARY KEY	Gerado na aplicação (Model HasUuids), não pelo banco
protocolo	VARCHAR(20)	NOT NULL, UNIQUE	Gerado por GerarProtocoloAction, formato SOL-{ano}-{sequencial:04d}
nome_solicitante	VARCHAR(150)	NOT NULL	Nome fictício, sem dados pessoais reais
categoria	categoria_solicitacao (enum nativo)	NOT NULL	CONSULTA, EXAME, VACINACAO, OUTRO
prioridade	prioridade_solicitacao (enum nativo)	NOT NULL	BAIXA, MEDIA, ALTA, URGENTE

status	status_solicitacao (enum nativo)	NOT NULL, default RECEBIDA	Ver máquina de estados em docs/contrato_api.md
descricao	TEXT	NOT NULL	Resumo da solicitação
justificativa_prioridade	TEXT	NULL	Obrigatório na validação de aplicação (StoreSolicitacaoRequest) e reforçado por CHECK no banco quando prioridade = 'URGENTE'
data_criacao	TIMESTAMPTZ	NOT NULL, default now()	Mapeado para CREATED_AT do Eloquent
data_atualizacao	TIMESTAMPTZ	NOT NULL, default now()	Mapeado para UPDATED_AT do Eloquent

Índices

Índice	Colunas	Motivo
idx_solicitacoes_status	status	Filtro por status (GET /solicitacoes?status=)
idx_solicitacoes_categoria	categoria	Filtro por categoria
idx_solicitacoes_prioridade	prioridade	Filtro por prioridade
idx_solicitacoes_criacao	data_criacao DESC	Ordenação padrão da listagem
idx_solicitacoes_filtros_combo	status, categoria, prioridade	Caso comum de múltiplos filtros combinados

Restrições de Integridade

```
CONSTRAINT chk_justificativa_urgente
CHECK (prioridade <> 'URGENTE' OR justificativa_prioridade IS NOT NULL)
```

Garantida no nível do banco — vale mesmo para inserts que não passem pela camada de validação do Laravel (ex.: scripts administrativos, migrações de dados futuras).

2. Arquitetura — Modelo C4

Arquitetura — Modelo C4

Nível 1 — Diagrama de Contexto

O AtendSaúde é tratado como um sistema único, visto de fora: um ator humano interage com ele via navegador. Não há integrações com sistemas externos de terceiros nesta versão (sem gateway de pagamento, SSO externo, etc.).

► *Diagrama Mermaid — visualize em mermaid.live, num Markdown viewer compatível (ex.: GitHub, VS Code) ou renderize antes de exportar o PDF final.*

```
C4Context
  title Diagrama de Contexto – AtendSaúde

  Person(usuario, "Atendente / Gestor", "Registra, consulta, filtra e atualiza o status de solicitações de atendimento")

  System(atendsaude, "AtendSaúde", "Permite registrar e acompanhar solicitações de atendimento de saúde pública, com máquina de estados de status e protocolo único")

  Rel(usuario, atendsaude, "Usa", "HTTPS")
```

Nível 2 — Diagrama de Contêineres

Decompõe o sistema em três contêineres, cada um em seu próprio serviço Docker, comunicando-se apenas pela rede interna atendsaude_net:

► *Diagrama Mermaid — visualize em mermaid.live, num Markdown viewer compatível (ex.: GitHub, VS Code) ou renderize antes de exportar o PDF final.*

```
C4Container
  title Diagrama de Contêineres – AtendSaúde

  Person(usuario, "Atendente / Gestor")

  System_Boundary(atendsaude, "AtendSaúde") {
    Container(frontend, "Frontend SPA", "React 18, TypeScript, Vite, Tailwind CSS", "Interface web: listagem/filtros, formulário de criação, modal de detalhes e transição de status")
    Container(backend, "API REST", "Laravel 11, PHP 8.3", "Expõe /api/v1: validação, máquina de estados, geração de protocolo, health check")
    ContainerDb(db, "Banco de Dados", "PostgreSQL 16", "Persiste solicitações (com enums nativos) e o contador de protocolo por ano")
  }

  Rel(usuario, frontend, "Acessa", "HTTPS, porta 5173")
  Rel(frontend, backend, "Consome API REST", "JSON/HTTPS, porta 8000")
  Rel(backend, db, "Lê e escreve", "SQL, porta 5432")
```

Responsabilidade de cada contêiner

Contêiner	Responsabilidade	Não faz
Frontend SPA	Renderização, estado de UI (loading/erro/vazio), validação client-side apenas como UX (nunca como fonte de verdade), tipagem dos contratos (src/types/solicitacao.ts)	Não acessa o banco diretamente; não reimplementa a máquina de estados como regra definitiva
API REST	Validação (FormRequests), regra de negócio (Actions), persistência via Eloquent, contrato versionado /api/v1	Não lida com apresentação; não conhece detalhes de UI
Banco de Dados	Persistência relacional, integridade referencial e de domínio (enums nativos, CHECK constraints)	Não contém lógica de aplicação além de constraints declarativas

Caminho de evolução (arquitetura de serviços por domínio)

Hoje o backend é um monólito modular único (app/Actions/Solicitacao/, app/Models/Solicitacao.php). Se o domínio crescer (ex.: múltiplas unidades de saúde, autenticação multi-perfil, notificações), o próximo passo natural seria:

1. Extrair o domínio de solicitações para um módulo isolado dentro do próprio monólito (app/Domain/Solicitacoes/), preparando fronteiras claras antes de qualquer split físico.
2. Somente então considerar um serviço separado (ex.: um serviço de notificações assíncronas), comunicando-se por eventos de domínio já emitidos internamente, minimizando o retrabalho de uma eventual extração para microserviço.

3. Contrato da API REST

Contrato da API REST — AtendSaúde

Espelha openapi.yaml (raiz do projeto) em formato legível. Em caso de divergência, openapi.yaml é a fonte de verdade.

Base URL: /api/v1 **Formato:** todas as respostas de sucesso são envelopadas em { "data": ... } (listagens incluem também meta e links), seguindo o padrão de API Resources do Laravel.

POST /solicitacoes

Cria uma nova solicitação. Status inicial sempre RECEBIDA; protocolo gerado automaticamente.

Parâmetros de corpo (JSON):

Campo	Tipo	Obrigatório	Observação
nome_solicitante	string (máx. 150)	sim	
categoria	enum	sim	CONSULTA, EXAME, VACINACAO, OUTRO
prioridade	enum	sim	BAIXA, MEDIA, ALTA, URGENTE
descricao	string	sim	
justificativa_prioridade	string \	null	condicional

201 Created

```
{
  "data": {
    "id": "b1f2e3a4-5678-90ab-cdef-1234567890ab",
    "protocolo": "SOL-2026-0001",
    "nome_solicitante": "Maria da Silva",
    "categoria": "CONSULTA",
    "prioridade": "ALTA",
    "status": "RECEBIDA",
    "descricao": "Encaminhamento para consulta cardiológica",
    "justificativa_prioridade": null,
    "data_criacao": "2026-09-17T13:00:00Z",
    "data_atualizacao": "2026-09-17T13:00:00Z"
  }
}
```

422 Unprocessable Entity (ex.: URGENTE sem justificativa)

```
{
  "message": "Os dados informados são inválidos.",
}
```

```
"errors": {
  "justificativa_prioridade": ["O campo justificativa da prioridade é obrigatório quando a prioridade é URGENTE."]
}
```

GET /solicitacoes

Lista solicitações com paginação e filtros.

Query params:

Param	Tipo	Default	Observação
page	integer	1	
per_page	integer	15	máx. 100
status	enum	—	filtro exato
categoria	enum	—	filtro exato
prioridade	enum	—	filtro exato

200 OK

```
{
  "data": [ { "...solicitação..." } ],
  "meta": { "current_page": 1, "per_page": 15, "total": 42, "last_page": 3 },
  "links": { "first": "...", "last": "...", "prev": null, "next": "..." }
}
```

GET /solicitacoes/{id}

Consulta os detalhes de uma solicitação. {id} é o UUID.

200 OK: objeto único, mesmo formato do POST.

404 Not Found

```
{ "message": "Solicitação não encontrada." }
```

PATCH /solicitacoes/{id}/status

Atualiza o status respeitando a máquina de estados abaixo. Regra centralizada em `App\Enums>StatusSolicitacao::transicoesPermitidas()`.

Status atual	Próximos status permitidos
--------------	----------------------------

RECEBIDA	EM_ANALISE, CANCELADA
EM_ANALISE	AGENDADA, CANCELADA
AGENDADA	CONCLUIDA, CANCELADA
CONCLUIDA	<i>(nenhum — status final)</i>
CANCELADA	<i>(nenhum — status final)</i>

Corpo:

```
{ "status": "EM_ANALISE" }
```

200 OK: solicitação atualizada, data_atualizacao renovado.

422 Unprocessable Entity (transição inválida ou status já final)

```
{
  "message": "Transição de status inválida.",
  "errors": { "status": ["Não é permitido mudar de CONCLUIDA para EM_ANALISE."] }
}
```

404 Not Found: solicitação inexistente.

GET /health

Health check da API e da conexão com o PostgreSQL.

200 OK

```
{ "status": "ok", "database": "connected", "timestamp": "2026-09-17T13:00:00Z" }
```

503 Service Unavailable (Postgres inacessível)

```
{ "status": "degraded", "database": "unavailable", "timestamp": "2026-09-17T13:00:00Z" }
```

4. Registros de Decisão Arquitetural (ADRs)

Registros de Decisão Arquitetural (ADRs) — AtendSaúde

Formato: Contexto → Decisão → Alternativas consideradas → Consequências. Todas as decisões abaixo foram validadas na Fase 0 (planejamento) antes da escrita de código e permanecem em vigor na versão atual do projeto.

ADR-001 — Service/Action Pattern para a máquina de estados

Status: Aceito

Contexto: a transição de status (RECEBIDA → EM_ANALISE → AGENDADA → CONCLUIDA/CANCELADA) é a regra de negócio mais crítica do domínio e a mais fácil de "vazar" para o Controller ao longo do tempo, tornando-a difícil de testar isoladamente.

Decisão: o mapa de transições permitidas vive em

`App\Enums>StatusSolicitacao::transicoesPermitidas()` (única fonte de verdade). Uma classe dedicada, `AtualizarStatusSolicitacaoAction`, consulta esse mapa, executa a mudança dentro de uma transação de banco e lança `TransicaoStatusInvalidaException` quando a transição é ilegal ou o status atual já é final.

Alternativas consideradas:

- *Regra dentro do Model (Eloquent observer/mutator):* rejeitada — acopla a regra ao ciclo de vida do Eloquent, dificultando testar a transição sem side effects de persistência.
- *Regra dentro do Controller:* rejeitada — é exatamente o anti-padrão que o edital pede para evitar ("Controllers não devem concentrar regras de negócio relevantes").

Consequências: o Controller vira apenas orquestração (`FormRequest` → `Action` → `Resource`); a regra é testável isoladamente com PHPUnit sem subir HTTP (ver `tests/Feature/Actions/AtualizarStatusSolicitacaoActionTest.php`).

ADR-002 — Geração atômica de protocolo

Status: Aceito

Contexto: cada solicitação precisa de um protocolo único e legível por humanos, no formato `SOL-{ano}-{sequencial:04d}`, sem colisão mesmo sob criação concorrente.

Decisão: uma tabela de apoio (`protocolo_sequences`, chave primária `ano`) é atualizada via `INSERT ... ON CONFLICT (ano) DO UPDATE SET ultimo_numero = ultimo_numero + 1 RETURNING ultimo_numero`, dentro da **mesma transação** que cria a solicitação (`CriarSolicitacaoAction`). Se a criação falhar, o incremento é revertido junto — sem "buracos" na numeração.

Alternativas consideradas:

- *Sequência nativa do Postgres* (`SERIAL`/`SEQUENCE`): rejeitada — não reseta por ano automaticamente sem lógica adicional, e o `nextval()` não é transacional (uma sequência consumida e depois revertida por rollback deixa buraco na numeração).
- *UUID como protocolo*: rejeitada — não é amigável para um operador humano (edital pede um código memorável/rastreável).

Consequências: garante unicidade e continuidade sob concorrência, ao custo de uma tabela e uma query extra por criação — trade-off aceitável dado o volume esperado do domínio.

ADR-003 — Enums nativos do PostgreSQL

Status: Aceito

Contexto: os campos `categoria`, `prioridade` e `status` têm um conjunto fechado e pequeno de valores válidos.

Decisão: `categoria_solicitacao`, `prioridade_solicitacao` e `status_solicitacao` são tipos ENUM nativos do Postgres (criados via `DB::statement` na migration), não VARCHAR com CHECK. No Laravel, são mapeados para PHP backed enums (`App\Enums\*`) via `cast` no Model.

Alternativas consideradas:

- `VARCHAR` + `CHECK` *constraint*: mais portátil entre SGBDs, mas realiza a mesma validação com um tipo menos expressivo — o schema por si só não documenta os valores possíveis ao inspecionar a tabela.

Consequências: integridade garantida no nível do banco mesmo para inserts fora da aplicação; ganho de auto-documentação do schema. Custo: menor portabilidade (enums do Postgres não migram automaticamente para outro SGBD), aceitável pois a stack já fixa PostgreSQL.

ADR-004 — Estratégia de containers (Docker Compose)

Status: Aceito

Contexto: o desafio exige execução integrada e reproduzível via Docker Compose.

Decisão: três serviços — db (`postgres:16-alpine`, com `healthcheck`), backend (Laravel via `php artisan serve`, atrás de um `entrypoint.sh` que aguarda o banco, gera a `APP_KEY`, roda migrations e seeders) e frontend (Vite dev server) — na mesma rede Docker `atendsaude_net`.

Alternativas consideradas:

- *PHP-FPM + Nginx no backend*: mais próximo de produção, mas adiciona um serviço e configuração extra sem benefício para o escopo de desenvolvimento/avaliação local. Registrado como ajuste necessário em `docs/deploy.md` para produção.

Consequências: setup mais simples para o avaliador (`docker compose up -d --build` e pronto), com a ressalva documentada de que `php artisan serve` não é recomendado para produção.

ADR-005 — Contrato tipado e desacoplamento frontend/backend

Status: Aceito

Contexto: o edital exige que o frontend consuma exclusivamente a API, com tipos TypeScript consistentes e sem uso indiscriminado de `any`.

Decisão: o contrato é versionado em `/api/v1` e documentado em `openapi.yaml` + `docs/contrato_api.md`. Os tipos TypeScript (`frontend/src/types/solicitacao.ts`) espelham manualmente os DTOs da API.

Alternativas consideradas:

- *Geração automática de tipos a partir do OpenAPI (ex.: ``openapi-typescript``):* avaliada e descartada para o escopo atual — adicionaria uma etapa de build extra e uma dependência de geração de código para um contrato com poucos endpoints, sem benefício proporcional ao esforço. Fica registrada como possível evolução caso a API cresça.

Consequências: exige disciplina manual para manter os tipos sincronizados com a API a cada mudança de contrato; aceitável dado o tamanho atual do domínio.

ADR-006 — Estratégia de testes

Status: Aceito

Contexto: o edital pede testes determinísticos e independentes de dados reais, tanto no backend quanto no frontend.

Decisão:

- **Backend:** testes de Feature (PHPUnit) rodando contra um banco Postgres **real e isolado** (`atendsaude_test`, configurado em `phpunit.xml`), não SQLite em memória — necessário porque as migrations usam `CREATE TYPE ... AS ENUM`, sintaxe específica do Postgres que o SQLite não suporta.
- **Frontend:** teste de componente (Vitest + Testing Library) sobre `FormularioSolicitacao`, validando comportamento (exibição condicional do campo de justificativa, payload enviado, exibição de erros de validação), não apenas snapshot ou cobertura de linha.

Alternativas consideradas:

- *SQLite em memória para testes de backend:* rejeitada pela incompatibilidade com enums nativos do Postgres (exigiria duplicar a lógica das migrations com um dialeto diferente do de produção, quebrando a garantia de que o teste reflete o schema real).

- *Testes E2E (Cypress/Playwright) no lugar do teste de componente:* considerados, mas não implementados neste ciclo por restrição de tempo — registrado como limitação conhecida em `README.md`.

Consequências: suíte de backend mais fiel ao ambiente de produção, ao custo de exigir um Postgres disponível (já satisfeito pelo próprio Docker Compose do projeto) em vez de um banco em memória mais rápido de inicializar.

5. Guia de Deploy

Guia de Deploy — AtendSaúde

Escopo: este guia é o objetivo de **Prioridade 2** do projeto (bônus para portfólio pessoal / live demo). A avaliação da seleção V-Lab (Prioridade 1) considera exclusivamente a execução local via Docker Compose descrita no README.md — nenhum passo abaixo é pré-requisito para a entrega do desafio. Até o deploy ser executado, o link de "Live Demo" no README permanece como placeholder.

Duas rotas práticas são descritas: **(A) VPS com Docker Compose** (mais controle, custo fixo baixo) e **(B) Render** (zero manutenção de servidor, adequado para uma demo de portfólio). Escolha uma — não são complementares.

Rota A — VPS + Docker Compose + Nginx + Certbot

A.1 Provisionar o servidor

```
# Numa VPS Ubuntu 24.04 nova (ex.: 2 vCPU / 4GB RAM)
sudo apt update && sudo apt install -y ca-certificates curl gnupg

# Instalar Docker Engine + Compose plugin (repositório oficial Docker)
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker
```

A.2 Clonar e configurar

```
git clone https://github.com/albertsantanadev/atendsaude.git /srv/atendsaude
cd /srv/atendsaude

cp .env.example .env
# Editar .env: gerar valores de produção reais para
# APP_ENV=production
# APP_DEBUG=false
# APP_URL=https://seu-dominio.com
# DB_PASSWORD=<senha forte, não a de desenvolvimento>
```

 Nunca reutilize a senha de desenvolvimento (postgres/postgres) em produção.

A.3 Criar o compose de produção

O docker-compose.yml do repositório é otimizado para desenvolvimento (php artisan serve, Vite dev server). Para produção, crie docker-compose.prod.yml ao lado dele:

```
# arquivo: docker-compose.prod.yml (a criar)
services:
  db:
    image: postgres:16-alpine
    restart: always
    environment:
      POSTGRES_DB: ${DB_DATABASE}
```

```

    POSTGRES_USER: ${DB_USERNAME}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
volumes:
  - atendsaude_pgdata_prod:/var/lib/postgresql/data
networks: [atendsaude_net]

backend:
  build:
    context: .
    dockerfile: docker/backend/Dockerfile.prod    # variante PHP-FPM (a criar)
  restart: always
  env_file: .env
  environment:
    DB_HOST: db
  networks: [atendsaude_net]

frontend:
  build:
    context: .
    dockerfile: docker/frontend/Dockerfile.prod    # variante multi-stage -> Nginx (a
criar)
  restart: always
  networks: [atendsaude_net]

nginx:
  image: nginx:alpine
  restart: always
  ports: ["80:80", "443:443"]
  volumes:
    - ./nginx/conf.d:/etc/nginx/conf.d:ro
    - certbot_www:/var/www/certbot
    - certbot_certs:/etc/letsencrypt
  depends_on: [frontend, backend]
  networks: [atendsaude_net]

volumes:
  atendsaude_pgdata_prod:
  certbot_www:
  certbot_certs:

networks:
  atendsaude_net:
    driver: bridge

```

*Dockerfile.prod do backend (PHP-FPM em vez de artisan serve) e do frontend (npm run build → nginx:alpine servindo /dist) ainda **não existem** no repositório — são o próximo passo antes do primeiro deploy real nesta rota.*

A.4 Subir e migrar

```
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml exec backend php artisan key:generate --force
docker compose -f docker-compose.prod.yml exec backend php artisan migrate --force
```

--force é obrigatório: o Artisan bloqueia migrate em APP_ENV=production sem essa flag, como proteção contra execução acidental.

A.5 TLS com Certbot

```
docker run --rm \
-v certbot_www:/var/www/certbot \
-v certbot_certs:/etc/letsencrypt \
certbot/certbot certonly --webroot -w /var/www/certbot \
-d seu-dominio.com --email voce@exemplo.com --agree-tos --non-interactive
```

Configurar renovação automática via cron (certbot renew) rodando a cada 12h.

A.6 Automação (CD)

.github/workflows/deploy.yml (já no repositório) executa os passos A.4 via SSH sempre que o workflow CI conclui com sucesso na branch main. Requer os secrets DEPLOY_HOST, DEPLOY_USER, DEPLOY_SSH_KEY configurados no GitHub.

Rota B — Render (managed, menor esforço)

Recomendada para uma demo pública rápida sem gerenciar servidor.

3. **Banco de dados:** criar um PostgreSQL gerenciado no Render (ou usar [Neon](#)). Copiar a connection string.
4. **Backend:** novo *Web Service* no Render, "Build and deploy from a Dockerfile", apontando para docker/backend/Dockerfile. Em *Environment*, configurar as variáveis (DB_CONNECTION=pgsql, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD extraídos da connection string, APP_KEY gerado localmente com php artisan key:generate --show, APP_ENV=production, APP_DEBUG=false).
5. **Migrations em produção:** no Render, usar o campo *Pre-Deploy Command* (ou um *Job* manual) com:

```
php artisan migrate --force
```

executado automaticamente a cada novo deploy, antes do serviço receber tráfego.

6. **Frontend:** novo *Static Site* no Render — *Build Command* npm run build (dentro de frontend/), *Publish Directory* frontend/dist. Variável de build VITE_API_BASE_URL apontando para a URL pública do backend (https://<backend>.onrender.com/api/v1).
 7. **Domínio e TLS:** Render provisiona HTTPS automaticamente para o domínio *.onrender.com; um domínio próprio pode ser apontado via CNAME nas configurações do serviço.
-

Gerenciamento de Variáveis de Ambiente em Produção

Variável	Origem em produção	Nunca fazer
APP_KEY	Gerada uma única vez (artisan key:generate --show) e fixada como secret	Regenerar a cada deploy (invalida sessões/dados criptografados existentes)
DB_PASSWORD	Secret do provedor (Render/GitHub Actions) ou gerenciador de secrets da VPS	Reutilizar a senha de desenvolvimento (postgres)
APP_DEBUG	false	Deixar true em produção — vaza stack traces nas respostas de erro
VITE_API_BASE_URL	Definida em build-time do frontend (Vite embute no bundle)	Esperar que funcione como variável de runtime — precisa de rebuild se a URL da API mudar

Próximos Passos Antes do Primeiro Deploy Real

8. Criar docker/backend/Dockerfile.prod (PHP-FPM) e docker/frontend/Dockerfile.prod (multi-stage → Nginx) — hoje só existem as variantes de desenvolvimento.
9. Decidir a Rota A ou B e provisionar a infraestrutura correspondente.
10. Substituir o placeholder de "Live Demo" no README.md pela URL real.
11. Adicionar monitoramento básico (uptime check em GET /api/v1/health) — ver observação em ADR/README sobre observabilidade não implementada.

